

Project #3: Decision Tree

Introduction

In machine learning, supervised learning is a common approach to learn from labeled data for making predictions. It's commonly categorized into classification and regression. Classification task is to predict a discrete label and regression task is to predict a contiguous value. In supervised learning, we need ways to split data into homogeneous groups. One of the useful ways called Information theory thus become important. Information theory measures the amount of uncertainty or impurity in our data, helping us to split data according to it. Inspired by information theory, decision tree is a supervised learning method that splits data based on a specific feature. Its aim is to maximize information gain (or minimize entropy). The goal of a decision tree is to create a model that can predict the labels correctly by learning rules that split the data into smaller, homogeneous groups.

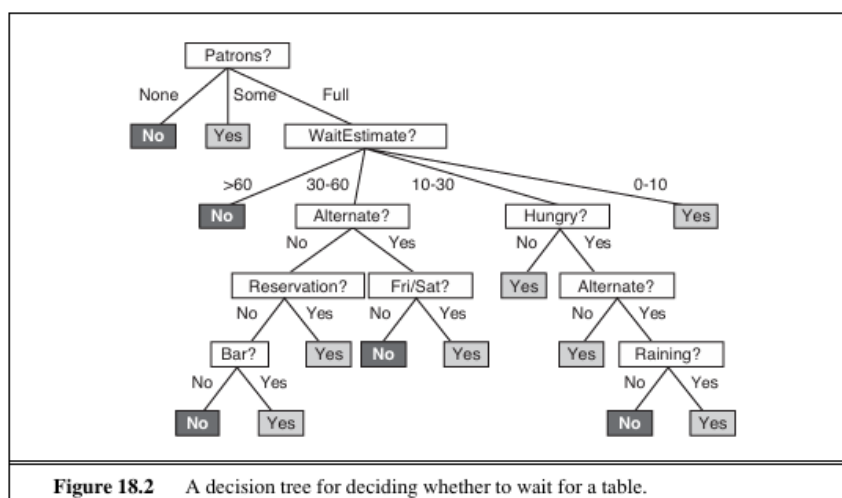


Figure 18.2 A decision tree for deciding whether to wait for a table.

(Figure 18.2. Artificial Intelligence. A Modern Approach. P699.)

Objectives

In this project, there's some jobs you need to finish. First, you need to use the code you wrote in project #1 to handle data preprocessing for project #3. Second, implement DecisionTree class in *model.py/model.hpp* and train it on the training data. Finally, generate the predict label of the testing data.

(Go check [*proj3_guildline_python.pdf*](#) or [*proj3_guildline_cpp.pdf*](#) for the further detail)

Dataset Description

The data for this project is real-word data for hospitalizing patients. Each patient is described by 77 *features* (F1~F77), together with a binary outcome. The project involves a training model with provided datasets (train_X.csv & train_y.csv) and final testing on an independent test set (test_X.csv & test_y.csv).

Note:

*In this project, we only provide test_X.csv for you. This means that you cannot directly use the test data to evaluate the model's performance. Therefore, we recommend using **K-fold cross-validation (from scratch)** to decide the best model and **predict the label** for the testing data.*

We will use your predict label to help you calculate the final prediction score.

Language:

You can implement your code by using either *Python3* or *C++*.

(just need to submit one of them)

Requirements:

1. Download the necessary library:

This part is **same** as project #1.

2. Preprocess the data

Use the code you wrote in the previous project to handle data preprocessing.

(You may need to modify your code for handling different categorical values and select features from now on)

3. Implement the detail of Decision Tree class:

We have written some start codes framework in *model.py/model.hpp*, please finish your implementation of decision tree according to it.

Note:

You don't have to follow the start codes in *model.py/model.hpp*, but we suggest you build your model by following the structure.

4. Print out your Tree structure

You need to write a function to print out your decision tree structure. The function should print out the tree only after the whole trained tree is generated.

Here is an example of tree structure:

In each row:

- [F<feature most recently split on>] [<number of 0s> 0 / <number of 1s> 1]

e.g. The following screenshot shows the first 3 layers of my trained decision tree.

```

[F2] [310 0 / 116 1]
Left:
  [F57] [206 0 / 111 1]
  Left:
    [F28] [41 0 / 56 1]
    Right:
      [F6] [165 0 / 55 1]
  Right:
    [F6] [104 0 / 5 1]
  Left:
    [F9] [87 0 / 1 1]
  Right:
    [F10] [17 0 / 4 1]

```

Note: the basic requirement is to print out the first three layers of your decision tree like the above screenshot. You can decide what additional information you want to print.

5. Predict the label of the testing data

Use your best model to predict the label of the testing data and upload it to E3. Please save the label like the screenshot below.

(The first column is index, and the second is label)

1	label
2	0
3	1
4	2
5	3
6	4
7	5
8	6
9	7
10	8
11	9
12	10
13	11
14	12
15	13
16	14
17	15
18	16
19	17
20	18

Note: if you don't save the label **like this**, you will **lose 5 pts** of the performance score in this project.

6. Do your implement **from scratch**

Do not use any machine learning libraries, except for the part we've already done for you in the framework.

Grading Policy (Total 110 pt includes bonus)

1. Report Spec (75%) (follow this format to write your report)

- A. Explain the concept and code detail of your Decision Tree model. (40 %)
- B. What is the problem if you use Information Gain to split data in your decision tree model? What is the solution to solve this issue? Please be precise and concise. (15%)
- C. Screenshot and paste your tree structure (10%)
(Please briefly explain what information you print in each row)
- D. Discussion (10%)
 - (a) Discuss your strategy to find the best model for the testing data in this project.
 - (b) Anything else you would like to discuss. (Optional)
- E. Bonus (10%)

In decision tree, overfitting is an important issue. Fortunately, here's some ways to solve the overfitting problem. One of the methods is pruning the decision tree.

The extra bonus here is implementing **pre-pruning** and **post-pruning** techniques. Explain their concept and code in detail.
(5 % for each)

2. Performance (25%)

For your result of performance, you'll get:

- 25/25: if scoring ≥ 0.62
- 20/25: if $0.62 > \text{scoring} \geq 0.57$
- 15/25: if $0.57 > \text{scoring} \geq 0.5$
- 0/25: if scoring < 0.5

$$\text{*Scoring} = 0.3 * \text{acc} + 0.35 * \text{f1 score} + 0.35 * \text{mcc}$$

Performance Metric	Definition
Recall ^a	$TP/(TP + FN)$
Precision ^b	$TP/(TP + FP)$
ACC	$(TP + TN)/(TP + TN + FP + FN)$
F1-score	$\frac{2 \times Recall \times Precision}{Recall + Precision}$
MCC	$\frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP) \times (TP + FN) \times (TN + FP) \times (TN + FN)}}$
AUC	Area under the ROC curve

TP, true positive; TN, true negative; FP, false positive; FN, false negative; MCC, Matthews correlation coefficient; ACC, accuracy; AUC, area under the curve; ROC, receiver operating characteristic.

^a Recall is equivalent to sensitivity in its definition.

^b Precision is equivalent to positive predictive value in its definition.

Note: We will use the predict label you upload to E3 to calculate your performance score. Also, be sure you **set up the random seed** of your best model to avoid controversy.

e.g.

```
import random
random.seed(0)
```

```
import numpy as np
np.random.seed(0)
```

Submission Deadline:

Deadline: 11/12 (Tue.) 11:59 p.m.

Submission format: (Remember to upload your predict label)

- You need to submit **all your code in one folder** together with your **report file, prediction label**, and zip them into one **.zip** file.
- Name your predict label as “pred_hw3_<yourStudentID>_<yourName>.csv”
e.g. “pred_hw3_312551120_陳柏憲.csv”
- Name your code folder as “<language>_hw3_<yourStudentID>_<yourName>”,
e.g. “Cpp_hw3_312551120_陳柏憲” or “Python_hw3_312551120_陳柏憲”
- Name your report file as “report_hw3_<yourStudentID>_<yourName>.pdf”,
e.g. “Report_hw3_312551120_陳柏憲.pdf”
- Name your .zip file as “ml_hw3_<yourStudentID>_<yourName>.zip”,
e.g. “ML_hw3_312551120_陳柏憲.zip”

(Uppercase or lowercase of the letters is fine)

Punishment Policy:

Except for what we mentioned in this file, please refer to [project_policy.pdf](#) we provided in project #1.

We will also run your code in this project, if we **cannot compile and run** your code, we will ask you to re-submit your code, and you will **lose some points** in the final score of this project. (You can tell us how to execute your code and where we should modify the path as the previous project do)

Also, in this project, if your performance metrics are very similar to your classmate's, we may suspect cheating and check your code very carefully. Maybe we will ask you and your classmate to explain it

in person. Please be aware!

Good luck~

TAs